

# Computer Science: The Part They Expect You to Just Know

*Python, Data Structures, and Algorithms for When the Interview Is Tomorrow*

Alan Li

New York University

# Contents

|           |                                                    |           |
|-----------|----------------------------------------------------|-----------|
| <b>I</b>  | <b>Programming Language</b>                        | <b>1</b>  |
| <b>1</b>  | <b>Python</b>                                      | <b>2</b>  |
| 1.1       | Data Types & Representation . . . . .              | 2         |
| 1.2       | Operators & Expressions . . . . .                  | 3         |
| 1.3       | Control Flow . . . . .                             | 4         |
| 1.4       | Functions . . . . .                                | 5         |
| 1.5       | Strings . . . . .                                  | 6         |
| 1.6       | Lists & Tuples . . . . .                           | 7         |
| 1.7       | Dictionaries & Sets . . . . .                      | 8         |
| 1.8       | File I/O & Exceptions . . . . .                    | 10        |
| 1.9       | Built-in Functions & Modules . . . . .             | 11        |
| <b>II</b> | <b>Data Structures &amp; Algorithms</b>            | <b>13</b> |
| <b>2</b>  | <b>Foundations</b>                                 | <b>14</b> |
| 2.1       | Algorithm Analysis & Asymptotic Notation . . . . . | 14        |
| 2.2       | Recursion . . . . .                                | 15        |
| <b>3</b>  | <b>Core Data Structures</b>                        | <b>16</b> |
| 3.1       | Arrays . . . . .                                   | 16        |
| 3.2       | Stacks . . . . .                                   | 16        |
| 3.3       | Queues . . . . .                                   | 17        |
| 3.4       | Linked Lists . . . . .                             | 17        |
| 3.5       | Trees . . . . .                                    | 18        |
| <b>4</b>  | <b>Sorting</b>                                     | <b>19</b> |
| 4.1       | Comparison-Based Sorting Algorithms . . . . .      | 19        |
| 4.2       | Lower Bound for Comparison Sorting . . . . .       | 21        |
| 4.3       | Sorting Algorithms Summary . . . . .               | 21        |
| <b>5</b>  | <b>Graph Algorithms</b>                            | <b>22</b> |
| 5.1       | Graph Basics . . . . .                             | 22        |
| 5.2       | Breadth-First Search (BFS) . . . . .               | 22        |
| 5.3       | Depth-First Search (DFS) & Applications . . . . .  | 23        |
| 5.4       | Topological Sort . . . . .                         | 24        |
| 5.5       | Strongly Connected Components (SCC) . . . . .      | 24        |
| 5.6       | Minimum Spanning Trees (MST) . . . . .             | 24        |
| 5.7       | Shortest Paths . . . . .                           | 25        |

|            |                                                     |           |
|------------|-----------------------------------------------------|-----------|
| <b>6</b>   | <b>Algorithm Design Paradigms</b>                   | <b>27</b> |
| 6.1        | Divide & Conquer . . . . .                          | 27        |
| 6.2        | Greedy Algorithms . . . . .                         | 27        |
| 6.3        | Dynamic Programming . . . . .                       | 28        |
| 6.4        | Matching . . . . .                                  | 29        |
| <b>7</b>   | <b>Computability &amp; Complexity</b>               | <b>30</b> |
| 7.1        | Computability . . . . .                             | 30        |
| 7.2        | NP-Completeness . . . . .                           | 31        |
| <b>III</b> | <b>Computer Systems</b>                             | <b>33</b> |
| <b>8</b>   | <b>Digital Foundations</b>                          | <b>34</b> |
| 8.1        | Binary & Number Systems . . . . .                   | 34        |
| 8.2        | Boolean Algebra & Logic Gates . . . . .             | 35        |
| 8.3        | Basic Circuits . . . . .                            | 36        |
| <b>9</b>   | <b>Computer Organization</b>                        | <b>37</b> |
| 9.1        | A Brief History of Computing . . . . .              | 37        |
| 9.2        | The Von Neumann Model . . . . .                     | 38        |
| 9.3        | Registers, ALU, and the Instruction Cycle . . . . . | 38        |
| <b>10</b>  | <b>Memory Hierarchy</b>                             | <b>39</b> |
| 10.1       | Memory Types . . . . .                              | 39        |
| 10.2       | The Memory Hierarchy and Cache . . . . .            | 39        |
| 10.3       | Why This Matters for ML/DS . . . . .                | 40        |
| <b>IV</b>  | <b>Computer Networking</b>                          | <b>41</b> |
| <b>11</b>  | <b>Network Fundamentals</b>                         | <b>42</b> |
| 11.1       | The Internet and Packet Switching . . . . .         | 42        |
| 11.2       | Layered Architecture . . . . .                      | 42        |
| <b>12</b>  | <b>Application Layer</b>                            | <b>44</b> |
| 12.1       | HTTP and the Web . . . . .                          | 44        |
| 12.2       | DNS . . . . .                                       | 45        |
| <b>13</b>  | <b>Transport Layer</b>                              | <b>46</b> |
| 13.1       | TCP vs. UDP . . . . .                               | 46        |
| 13.2       | Ports and Sockets . . . . .                         | 47        |
| <b>14</b>  | <b>Network Layer</b>                                | <b>48</b> |
| 14.1       | IP Addressing and Subnets . . . . .                 | 48        |
| 14.2       | Routing Overview . . . . .                          | 49        |
| 14.3       | IPv6 Basics . . . . .                               | 49        |
| <b>15</b>  | <b>Link Layer (Brief)</b>                           | <b>50</b> |
| 15.1       | MAC Addresses and Ethernet . . . . .                | 50        |
| 15.2       | ARP and DHCP . . . . .                              | 50        |

Part I

Programming Language

# Chapter 1

# Python

Python is a high-level, interpreted, dynamically-typed language. This chapter is a structured reference for core language features, organized for quick lookup. Code examples use Python 3.

## 1.1 Data Types & Representation

### Primitive Types

| Type     | Keyword              | Description                                                                | Example                                |
|----------|----------------------|----------------------------------------------------------------------------|----------------------------------------|
| Integer  | <code>int</code>     | Arbitrary precision whole number                                           | 42, -7, 0b1010, 0xFF                   |
| Float    | <code>float</code>   | 64-bit IEEE 754 double                                                     | 3.14, 2.5e-3                           |
| Complex  | <code>complex</code> | Real + imaginary part                                                      | 3+4j, <code>complex(1,2)</code>        |
| Boolean  | <code>bool</code>    | Subclass of <code>int</code> ; <code>True</code> =1, <code>False</code> =0 | <code>True</code> , <code>False</code> |
| String   | <code>str</code>     | Immutable Unicode sequence                                                 | "hello", 'world'                       |
| NoneType | <code>None</code>    | Null/absence of value                                                      | <code>None</code>                      |

#### Remark.

Integers in Python have arbitrary precision — no integer overflow. Booleans are a subclass of `int`: `True + True == 2`.

### Type Conversion Functions

| Function                  | Converts to   | Notes                                                              |
|---------------------------|---------------|--------------------------------------------------------------------|
| <code>int(x)</code>       | Integer       | <code>int(3.9) → 3</code> (truncates); <code>int("42") → 42</code> |
| <code>float(x)</code>     | Float         | <code>float("3.14") → 3.14</code>                                  |
| <code>str(x)</code>       | String        | <code>str(42) → "42"</code>                                        |
| <code>bool(x)</code>      | Boolean       | <code>False</code> for: 0, "", [], {}, <code>None</code>           |
| <code>complex(r,i)</code> | Complex       | <code>complex(3,4) → (3+4j)</code>                                 |
| <code>bin(n)</code>       | Binary string | <code>bin(10) → "0b1010"</code>                                    |
| <code>hex(n)</code>       | Hex string    | <code>hex(255) → "0xff"</code>                                     |
| <code>ord(c)</code>       | Unicode int   | <code>ord("A") → 65</code>                                         |
| <code>chr(n)</code>       | Character     | <code>chr(65) → "A"</code>                                         |

## Data Representation

### Definition 1.1.1: Bit Width

An  $n$ -bit value can represent  $2^n$  distinct states.

| Bits | Range (unsigned) | Common use           |
|------|------------------|----------------------|
| 1    | 0–1              | boolean flag         |
| 8    | 0–255            | byte, ASCII char     |
| 16   | 0–65 535         | short integer        |
| 32   | 0–4 294 967 295  | int (C)              |
| 64   | 0– $2^{64} - 1$  | long, float (Python) |

## 1.2 Operators & Expressions

### Arithmetic Operators

| Op | Name           | Example                                           |
|----|----------------|---------------------------------------------------|
| +  | Addition       | $3 + 4 \rightarrow 7$                             |
| -  | Subtraction    | $7 - 3 \rightarrow 4$                             |
| *  | Multiplication | $3 * 4 \rightarrow 12$                            |
| /  | True division  | $7 / 2 \rightarrow 3.5$                           |
| // | Floor division | $7 // 2 \rightarrow 3$ ; $-7 // 2 \rightarrow -4$ |
| %  | Modulo         | $7 \% 3 \rightarrow 1$                            |
| ** | Exponentiation | $2 ** 3 \rightarrow 8$                            |

### Comparison & Logical Operators

| Op     | Name                  | Notes                                                                    |
|--------|-----------------------|--------------------------------------------------------------------------|
| ==     | Equal                 | Value equality                                                           |
| !=     | Not equal             |                                                                          |
| <      | Less than             |                                                                          |
| >      | Greater than          |                                                                          |
| <=     | Less than or equal    |                                                                          |
| >=     | Greater than or equal |                                                                          |
| and    | Logical AND           | Short-circuit: returns first falsy value or last value                   |
| or     | Logical OR            | Short-circuit: returns first truthy value or last value                  |
| not    | Logical NOT           | Unary; <code>not True</code> $\rightarrow$ <code>False</code>            |
| in     | Membership            | <code>"a" in "abc"</code> $\rightarrow$ <code>True</code>                |
| not in | Non-membership        |                                                                          |
| is     | Identity              | Same object in memory (not same value)                                   |
| is not | Non-identity          | Use <code>is</code> for <code>None</code> checks: <code>x is None</code> |

## Compound Assignment Operators

| Op                  | Equivalent             | Op                                                    |
|---------------------|------------------------|-------------------------------------------------------|
| <code>x += n</code> | <code>x = x + n</code> | <code>x /= n</code> $\equiv$ <code>x = x / n</code>   |
| <code>x -= n</code> | <code>x = x - n</code> | <code>x %= n</code> $\equiv$ <code>x = x % n</code>   |
| <code>x *= n</code> | <code>x = x * n</code> | <code>x **= n</code> $\equiv$ <code>x = x ** n</code> |
| <code>x /= n</code> | <code>x = x / n</code> |                                                       |

**Remark.**

**Operator precedence** (high to low): `**` > unary `-/+` > `*` / `//` / `%` > `+` / `-` > `<` / `>` / `<=` / `>=` > `==` / `!=` > `not` > `and` > `or`. Use parentheses when in doubt.

## 1.3 Control Flow

### Conditionals

```
if condition:
    ...
elif another_condition:
    ...
else:
    ...

# Ternary expression
value = x if condition else y
```

### While Loop

```
while condition:
    ...
    if some_case:
        break          # exit loop immediately
    if other_case:
        continue       # skip to next iteration
else:
    ...                 # runs if loop completed without break
```

### For Loop

```
for item in iterable:
    ...

for i in range(start, stop, step):    # range(5) = 0,1,2,3,4
    ...
```

```

for i, val in enumerate(seq, start=0):
    ...

for a, b in zip(seq1, seq2):           # parallel iteration
    ...

```

## Comprehensions

| Type      | Syntax                                    | Returns          |
|-----------|-------------------------------------------|------------------|
| List      | <code>[expr for x in iter if cond]</code> | list             |
| Dict      | <code>{k: v for x in iter if cond}</code> | dict             |
| Set       | <code>{expr for x in iter if cond}</code> | set              |
| Generator | <code>(expr for x in iter if cond)</code> | generator (lazy) |

### Example 1.3.1

```

squares = [x**2 for x in range(10)] → [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
even_sq = {x**2 for x in range(10) if x % 2 == 0} → {0, 4, 16, 36, 64}

```

## 1.4 Functions

### Definition 1.4.1: Function

A reusable, named block of code defined with **def**. Returns **None** implicitly if no **return** statement is reached. Multiple values can be returned as a tuple.

### Parameter Types

| Type                | Syntax                       | Example call                                   |
|---------------------|------------------------------|------------------------------------------------|
| Positional          | <code>def f(a, b)</code>     | <code>f(1, 2)</code>                           |
| Default value       | <code>def f(a, b=0)</code>   | <code>f(1)</code>                              |
| Variable positional | <code>def f(*args)</code>    | <code>f(1,2,3) ⇒ args=(1,2,3)</code>           |
| Variable keyword    | <code>def f(**kwargs)</code> | <code>f(x=1,y=2) ⇒ kwargs={'x':1,'y':2}</code> |
| Keyword-only        | <code>def f(*, a)</code>     | <code>f(a=1)</code> (must be named)            |
| Positional-only     | <code>def f(a, b, /)</code>  | <code>f(1, 2)</code> (cannot be named)         |

#### Remark.

Order of parameters: positional → default → **\*args** → keyword-only → **\*\*kwargs**.



## Scope: LEGB Rule

| Level     | Description                        | Example                                                    |
|-----------|------------------------------------|------------------------------------------------------------|
| Local     | Inside the current function        | variable defined in <code>def</code>                       |
| Enclosing | Outer function in nested functions | closure variable                                           |
| Global    | Module-level (top of file)         | <code>global x</code> to write                             |
| Built-in  | Python's pre-defined names         | <code>len</code> , <code>print</code> , <code>range</code> |

## Lambda Functions

```
f = lambda x, y: x + y          # anonymous function, single expression
sorted(items, key=lambda x: x[1])
```

## 1.5 Strings

Strings are *immutable* sequences of Unicode characters. Indexing uses `s[i]` (0-based); slicing uses `s[start:stop:step]`.

### String Methods — Case

| Method                     | Description           | Example                                            |
|----------------------------|-----------------------|----------------------------------------------------|
| <code>.lower()</code>      | All lowercase         | <code>"Hello".lower() → "hello"</code>             |
| <code>.upper()</code>      | All uppercase         | <code>"hello".upper() → "HELLO"</code>             |
| <code>.title()</code>      | Title Case            | <code>"hello world".title() → "Hello World"</code> |
| <code>.capitalize()</code> | Capitalize first char | <code>"hello".capitalize() → "Hello"</code>        |
| <code>.swapcase()</code>   | Swap all cases        | <code>"Hello".swapcase() → "hELLO"</code>          |

### String Methods — Search & Check

| Method                      | Description                                                  | Returns           |
|-----------------------------|--------------------------------------------------------------|-------------------|
| <code>.find(s)</code>       | Index of first <code>s</code> ; <code>-1</code> if not found | <code>int</code>  |
| <code>.rfind(s)</code>      | Index of last <code>s</code>                                 | <code>int</code>  |
| <code>.index(s)</code>      | Like <code>find</code> but raises <code>ValueError</code>    | <code>int</code>  |
| <code>.count(s)</code>      | Count non-overlapping occurrences of <code>s</code>          | <code>int</code>  |
| <code>.startswith(s)</code> | True if string starts with <code>s</code>                    | <code>bool</code> |
| <code>.endswith(s)</code>   | True if string ends with <code>s</code>                      | <code>bool</code> |
| <code>.isdigit()</code>     | All chars are digits                                         | <code>bool</code> |
| <code>.isalpha()</code>     | All chars are letters                                        | <code>bool</code> |
| <code>.isalnum()</code>     | All chars are letters or digits                              | <code>bool</code> |
| <code>.isspace()</code>     | All chars are whitespace                                     | <code>bool</code> |
| <code>.isupper()</code>     | All cased chars are uppercase                                | <code>bool</code> |
| <code>.islower()</code>     | All cased chars are lowercase                                | <code>bool</code> |

## String Methods — Strip & Replace

| Method                             | Description                                                       |
|------------------------------------|-------------------------------------------------------------------|
| <code>.strip()</code>              | Remove leading and trailing whitespace                            |
| <code>.lstrip()</code>             | Remove leading whitespace                                         |
| <code>.rstrip()</code>             | Remove trailing whitespace                                        |
| <code>.strip(chars)</code>         | Remove any chars in <code>chars</code> from both ends             |
| <code>.replace(old, new)</code>    | Replace all occurrences of <code>old</code> with <code>new</code> |
| <code>.replace(old, new, n)</code> | Replace at most <code>n</code> occurrences                        |

## String Methods — Split & Join

| Method                      | Description                         | Example                                        |
|-----------------------------|-------------------------------------|------------------------------------------------|
| <code>.split()</code>       | Split on whitespace                 | <code>"a b c".split() → ['a', 'b', 'c']</code> |
| <code>.split(sep)</code>    | Split on <code>sep</code>           | <code>"a,b".split(",") → ['a', 'b']</code>     |
| <code>.split(sep, n)</code> | Split at most <code>n</code> times  |                                                |
| <code>.rsplit(sep)</code>   | Split from right                    |                                                |
| <code>.splitlines()</code>  | Split on line boundaries            |                                                |
| <code>sep.join(iter)</code> | Join iterable with <code>sep</code> | <code>", ".join(['a', 'b']) → "a, b"</code>    |

## String Formatting

| Style                  | Syntax                                    | Example                    |
|------------------------|-------------------------------------------|----------------------------|
| % formatting           | <code>"%s is %d" % (name, age)</code>     | <code>"Alice is 25"</code> |
| <code>.format()</code> | <code>"{} is {}".format(name, age)</code> | <code>"Alice is 25"</code> |
| f-string (3.6+)        | <code>f"{name} is {age}"</code>           | <code>"Alice is 25"</code> |

| Format spec          | Meaning             | Example                                    |
|----------------------|---------------------|--------------------------------------------|
| <code>:.2f</code>    | 2 decimal places    | <code>f"{3.14159:.2f}" → "3.14"</code>     |
| <code>:10</code>     | Width 10            | <code>f"{42:10}" → " 42"</code>            |
| <code>:&lt;10</code> | Left-align width 10 | <code>f"{42:&lt;10}" → "42 "</code>        |
| <code>:^10</code>    | Center width 10     |                                            |
| <code>: ,</code>     | Thousands separator | <code>f"{1000000:,}" → "1,000,000"</code>  |
| <code>:e</code>      | Scientific notation | <code>f"{0.001:e}" → "1.000000e-03"</code> |

## 1.6 Lists & Tuples

### Definition 1.6.1: List

An ordered, *mutable* sequence. Elements can be of mixed types. Backed by a dynamic array; random access in  $O(1)$ , append in amortized  $O(1)$ .

## List Methods

| Method                           | Description                                       | Time          |
|----------------------------------|---------------------------------------------------|---------------|
| <code>.append(x)</code>          | Add <code>x</code> to end                         | $O(1)^*$      |
| <code>.extend(iter)</code>       | Add all items from iterable                       | $O(k)$        |
| <code>.insert(i, x)</code>       | Insert <code>x</code> at index <code>i</code>     | $O(n)$        |
| <code>.remove(x)</code>          | Remove first occurrence of <code>x</code>         | $O(n)$        |
| <code>.pop()</code>              | Remove and return last element                    | $O(1)^*$      |
| <code>.pop(i)</code>             | Remove and return element at index <code>i</code> | $O(n)$        |
| <code>.index(x)</code>           | Index of first occurrence of <code>x</code>       | $O(n)$        |
| <code>.count(x)</code>           | Count occurrences of <code>x</code>               | $O(n)$        |
| <code>.sort(key, reverse)</code> | Sort in place (Timsort)                           | $O(n \log n)$ |
| <code>.reverse()</code>          | Reverse in place                                  | $O(n)$        |
| <code>.copy()</code>             | Shallow copy                                      | $O(n)$        |
| <code>.clear()</code>            | Remove all items                                  | $O(n)$        |

### Remark.

\* Amortized. Use `sorted(lst)` to get a new sorted list without modifying the original. `lst[start:stop:step]` slices return a new list.

## List vs. Tuple

| Feature     | List                   | Tuple                                              |
|-------------|------------------------|----------------------------------------------------|
| Syntax      | <code>[1, 2, 3]</code> | <code>(1, 2, 3)</code>                             |
| Mutable     | Yes                    | No                                                 |
| Hashable    | No                     | Yes (if all elements hashable)                     |
| Performance | Slightly slower        | Slightly faster                                    |
| Typical use | Mutable collections    | Fixed records, dict keys, multiple return values   |
| Methods     | 11 methods             | Only <code>.count()</code> , <code>.index()</code> |

### Remark.

A single-element tuple requires a trailing comma: `(1,)` not `(1)`.

## 1.7 Dictionaries & Sets

### Definition 1.7.1: Dictionary

An unordered (insertion-ordered since Python 3.7) mapping of *hashable* keys to values. Implemented as a hash table; average  $O(1)$  access, insertion, and deletion.

## Dictionary Methods

| Method / Operation                      | Description                                          |
|-----------------------------------------|------------------------------------------------------|
| <code>d[key]</code>                     | Access value; raises <code>KeyError</code> if absent |
| <code>d.get(key)</code>                 | Access value; returns <code>None</code> if absent    |
| <code>d.get(key, default)</code>        | Access value with fallback                           |
| <code>d[key] = val</code>               | Set or update key                                    |
| <code>del d[key]</code>                 | Delete key                                           |
| <code>key in d</code>                   | Membership test for key                              |
| <code>d.keys()</code>                   | View of all keys                                     |
| <code>d.values()</code>                 | View of all values                                   |
| <code>d.items()</code>                  | View of all (key, value) pairs                       |
| <code>d.update(other)</code>            | Merge <code>other</code> dict into <code>d</code>    |
| <code>d.pop(key)</code>                 | Remove key and return its value                      |
| <code>d.pop(key, default)</code>        | Remove key with fallback                             |
| <code>d.popitem()</code>                | Remove and return last (key, val) pair               |
| <code>d.setdefault(key, default)</code> | Get value; set to default if key absent              |
| <code>d.clear()</code>                  | Remove all items                                     |
| <code>d.copy()</code>                   | Shallow copy                                         |
| <code>d   other</code>                  | Merge (Python 3.9+); creates new dict                |

### Definition 1.7.2: Set

An unordered collection of unique, hashable elements. Backed by a hash table; average  $O(1)$  insertion, deletion, and membership test. `set()` creates an empty set; `{}` creates an empty *dict*.

## Set Operations

| Operator     | Method                                 | Description                       |
|--------------|----------------------------------------|-----------------------------------|
| $A \mid B$   | <code>A.union(B)</code>                | All elements in $A$ or $B$        |
| $A \& B$     | <code>A.intersection(B)</code>         | Elements in both $A$ and $B$      |
| $A - B$      | <code>A.difference(B)</code>           | Elements in $A$ but not $B$       |
| $A \wedge B$ | <code>A.symmetric_difference(B)</code> | Elements in exactly one of $A, B$ |
| $A \leq B$   | <code>A.issubset(B)</code>             | $A \subseteq B$                   |
| $A \geq B$   | <code>A.issuperset(B)</code>           | $A \supseteq B$                   |
| $A < B$      | —                                      | $A \subsetneq B$ (proper subset)  |

## Set Methods

| Method                     | Description                                                    |
|----------------------------|----------------------------------------------------------------|
| <code>.add(x)</code>       | Add <code>x</code> ; no-op if already present                  |
| <code>.remove(x)</code>    | Remove <code>x</code> ; raises <code>KeyError</code> if absent |
| <code>.discard(x)</code>   | Remove <code>x</code> ; no-op if absent                        |
| <code>.pop()</code>        | Remove and return an arbitrary element                         |
| <code>.update(iter)</code> | Add all elements from iterable                                 |
| <code>.clear()</code>      | Remove all elements                                            |
| <code>.copy()</code>       | Shallow copy                                                   |

## 1.8 File I/O & Exceptions

### File Modes

| Mode | Name       | Behaviour                                                       |
|------|------------|-----------------------------------------------------------------|
| 'r'  | Read       | Default. Fails if file doesn't exist.                           |
| 'w'  | Write      | Creates file or truncates existing.                             |
| 'a'  | Append     | Creates file or appends to existing.                            |
| 'x'  | Create     | Creates file; raises <code>FileExistsError</code> if it exists. |
| 'b'  | Binary     | Modifier; e.g. 'rb', 'wb'.                                      |
| '+'  | Read+Write | Modifier; e.g. 'r+', 'w+'.                                      |

### File Methods

| Method                          | Description                                                   |
|---------------------------------|---------------------------------------------------------------|
| <code>.read()</code>            | Read entire file as one string                                |
| <code>.read(n)</code>           | Read at most <code>n</code> characters                        |
| <code>.readline()</code>        | Read one line (including <code>\n</code> )                    |
| <code>.readlines()</code>       | Read all lines into a list                                    |
| <code>.write(s)</code>          | Write string <code>s</code> ; returns number of chars written |
| <code>.writelines(lines)</code> | Write list of strings (no newlines added)                     |
| <code>.close()</code>           | Close the file (always call or use <code>with</code> )        |
| <code>.seek(pos)</code>         | Move file pointer to byte position                            |
| <code>.tell()</code>            | Return current file pointer position                          |

#### Remark.

Use the `with` statement — it guarantees the file is closed even if an exception occurs: `with open("file.txt") as f: ...`

### Exception Handling

```

try:
    risky_operation()
except ValueError as e:           # catch specific exception
    handle(e)
except (TypeError, KeyError):    # catch multiple types
    ...
except Exception as e:          # catch all (avoid)
    ...
else:
    ...                          # runs only if NO exception was raised
finally:
    ...                          # always runs (cleanup)

```

## Common Built-in Exceptions

| Exception         | Raised when                                                |
|-------------------|------------------------------------------------------------|
| ValueError        | Correct type but invalid value ( <code>int("abc")</code> ) |
| TypeError         | Wrong type ( <code>"1" + 1</code> )                        |
| IndexError        | Sequence index out of range                                |
| KeyError          | Dict key not found                                         |
| AttributeError    | Object has no such attribute                               |
| NameError         | Variable or name not defined                               |
| ZeroDivisionError | Division or modulo by zero                                 |
| FileNotFoundError | File or directory not found                                |
| ImportError       | Module cannot be imported                                  |
| StopIteration     | Iterator is exhausted                                      |
| RecursionError    | Maximum recursion depth exceeded                           |

## 1.9 Built-in Functions & Modules

### Frequently Used Built-ins

| Function                            | Description                                           | Example                                     |
|-------------------------------------|-------------------------------------------------------|---------------------------------------------|
| <code>len(x)</code>                 | Length of sequence/collection                         | <code>len([1,2,3]) → 3</code>               |
| <code>range(stop)</code>            | Range <code>[0, stop)</code>                          | <code>range(5)</code>                       |
| <code>range(start,stop,step)</code> | Range with step                                       | <code>range(0,10,2)</code>                  |
| <code>type(x)</code>                | Type of <code>x</code>                                | <code>type(42) → &lt;class 'int'&gt;</code> |
| <code>isinstance(x, T)</code>       | Check if <code>x</code> is instance of <code>T</code> | <code>isinstance(42, int)</code>            |
| <code>abs(x)</code>                 | Absolute value                                        | <code>abs(-5) → 5</code>                    |
| <code>round(x, n)</code>            | Round to <code>n</code> decimal places                | <code>round(3.14159, 2) → 3.14</code>       |
| <code>min(*args)</code>             | Minimum value                                         | <code>min(3,1,2) → 1</code>                 |
| <code>max(*args)</code>             | Maximum value                                         | <code>max(3,1,2) → 3</code>                 |
| <code>sum(iter)</code>              | Sum of iterable                                       | <code>sum([1,2,3]) → 6</code>               |
| <code>sorted(iter)</code>           | New sorted list                                       | <code>sorted([3,1,2]) → [1,2,3]</code>      |
| <code>reversed(iter)</code>         | Reversed iterator                                     |                                             |
| <code>enumerate(iter)</code>        | <code>(index, value)</code> pairs                     |                                             |
| <code>zip(*iters)</code>            | Parallel iteration tuples                             |                                             |
| <code>map(f, iter)</code>           | Apply <code>f</code> to each element                  |                                             |
| <code>filter(f, iter)</code>        | Elements where <code>f</code> is truthy               |                                             |
| <code>any(iter)</code>              | True if any element is truthy                         |                                             |
| <code>all(iter)</code>              | True if all elements are truthy                       |                                             |
| <code>id(x)</code>                  | Memory address of object                              |                                             |
| <code>hash(x)</code>                | Hash value (for hashable objects)                     |                                             |

**math Module**

| Name                             | Description                   | Value / Example                      |
|----------------------------------|-------------------------------|--------------------------------------|
| <code>math.pi</code>             | $\pi$                         | $\approx 3.14159$                    |
| <code>math.e</code>              | $e$                           | $\approx 2.71828$                    |
| <code>math.inf</code>            | Positive infinity             |                                      |
| <code>math.sqrt(x)</code>        | $\sqrt{x}$                    | <code>math.sqrt(9) → 3.0</code>      |
| <code>math.floor(x)</code>       | $\lfloor x \rfloor$           | <code>math.floor(3.9) → 3</code>     |
| <code>math.ceil(x)</code>        | $\lceil x \rceil$             | <code>math.ceil(3.1) → 4</code>      |
| <code>math.log(x)</code>         | $\ln x$                       | natural log                          |
| <code>math.log(x, b)</code>      | $\log_b x$                    | <code>math.log(8, 2) → 3.0</code>    |
| <code>math.log2(x)</code>        | $\log_2 x$                    |                                      |
| <code>math.log10(x)</code>       | $\log_{10} x$                 |                                      |
| <code>math.exp(x)</code>         | $e^x$                         |                                      |
| <code>math.pow(x, y)</code>      | $x^y$ (returns float)         |                                      |
| <code>math.factorial(n)</code>   | $n!$                          | <code>math.factorial(5) → 120</code> |
| <code>math.gcd(a, b)</code>      | Greatest common divisor       | <code>math.gcd(12, 8) → 4</code>     |
| <code>math.cos/sin/tan(x)</code> | Trig functions (radians)      |                                      |
| <code>math.degrees(r)</code>     | Radians $\rightarrow$ degrees |                                      |
| <code>math.radians(d)</code>     | Degrees $\rightarrow$ radians |                                      |

**random Module**

| Function                              | Description                                    |
|---------------------------------------|------------------------------------------------|
| <code>random.random()</code>          | Float in $[0.0, 1.0)$                          |
| <code>random.uniform(a, b)</code>     | Float in $[a, b]$                              |
| <code>random.randint(a, b)</code>     | Integer in $[a, b]$ (inclusive both ends)      |
| <code>random.randrange(n)</code>      | Integer in $[0, n)$                            |
| <code>random.choice(seq)</code>       | Random element from non-empty sequence         |
| <code>random.choices(seq, k=n)</code> | $n$ random elements with replacement           |
| <code>random.sample(seq, k)</code>    | $k$ unique random elements without replacement |
| <code>random.shuffle(seq)</code>      | Shuffle sequence in place                      |
| <code>random.seed(n)</code>           | Set random seed for reproducibility            |

## Part II

# Data Structures & Algorithms



# Chapter 2

## Foundations

### 2.1 Algorithm Analysis & Asymptotic Notation

#### Definition 2.1.1: Asymptotic Upper Bound ( $O$ )

$f(n) = O(g(n))$  if there exist constants  $c > 0$  and  $n_0 \geq 1$  such that for all  $n \geq n_0$ :

$$f(n) \leq c \cdot g(n)$$

Informally:  $f$  grows *no faster* than  $g$  asymptotically.

#### Definition 2.1.2: Asymptotic Lower Bound ( $\Omega$ )

$f(n) = \Omega(g(n))$  if there exist constants  $c > 0$  and  $n_0 \geq 1$  such that for all  $n \geq n_0$ :

$$f(n) \geq c \cdot g(n)$$

Informally:  $f$  grows *at least as fast* as  $g$ .

#### Definition 2.1.3: Tight Bound ( $\Theta$ )

$f(n) = \Theta(g(n))$  if  $f(n) = O(g(n))$  and  $f(n) = \Omega(g(n))$ . Informally:  $f$  and  $g$  have the *same* asymptotic growth rate.

### Complexity Hierarchy

| Class         | Name         | Example algorithm                           |
|---------------|--------------|---------------------------------------------|
| $O(1)$        | Constant     | Array access, hash table lookup             |
| $O(\log n)$   | Logarithmic  | Binary search                               |
| $O(n)$        | Linear       | Linear scan, linear search                  |
| $O(n \log n)$ | Linearithmic | Merge sort, Heap sort                       |
| $O(n^2)$      | Quadratic    | Insertion sort (worst), Bubble sort         |
| $O(n^3)$      | Cubic        | Naive matrix multiplication, Floyd-Warshall |
| $O(2^n)$      | Exponential  | Brute-force subset enumeration              |
| $O(n!)$       | Factorial    | Brute-force TSP                             |

**Remark.**

Growth order:  $1 \ll \log n \ll \sqrt{n} \ll n \ll n \log n \ll n^2 \ll n^3 \ll 2^n \ll n!$

## Rules for Computing Complexity

- **Sum rule:**  $O(f) + O(g) = O(\max(f, g))$  — keep the dominant term.
- **Product rule:**  $O(f) \cdot O(g) = O(f \cdot g)$  — nested loops multiply.
- **Drop constants:**  $O(5n) = O(n)$ .
- **Drop lower-order terms:**  $O(n^2 + n) = O(n^2)$ .
- **Logarithm base:**  $O(\log_a n) = O(\log_b n)$  — base doesn't matter.

## 2.2 Recursion

### Definition 2.2.1: Recurrence Relation

An equation defining  $T(n)$  in terms of  $T$  on smaller inputs. Typically  $T(n) = aT(n/b) + f(n)$  for divide-and-conquer algorithms, where  $a$  = number of subproblems,  $b$  = factor by which input shrinks,  $f(n)$  = cost outside recursive calls.

### Theorem 2.2.2: Master Theorem

Let  $T(n) = aT(n/b) + f(n)$  with  $a \geq 1$ ,  $b > 1$ . Let  $d = \log_b a$ .

1. If  $f(n) = O(n^{d-\epsilon})$  for some  $\epsilon > 0$ :  $T(n) = \Theta(n^d)$
2. If  $f(n) = \Theta(n^d)$ :  $T(n) = \Theta(n^d \log n)$
3. If  $f(n) = \Omega(n^{d+\epsilon})$  and regularity holds:  $T(n) = \Theta(f(n))$

### Example 2.2.1

**Merge Sort:**  $T(n) = 2T(n/2) + O(n)$ . Here  $a = 2$ ,  $b = 2$ ,  $d = \log_2 2 = 1$ . Since  $f(n) = O(n) = O(n^1) = \Theta(n^d)$ , Case 2 applies:  $T(n) = \Theta(n \log n)$ .

**Binary Search:**  $T(n) = T(n/2) + O(1)$ . Here  $a = 1$ ,  $b = 2$ ,  $d = \log_2 1 = 0$ . Since  $f(n) = O(1) = \Theta(n^0) = \Theta(n^d)$ , Case 2 applies:  $T(n) = \Theta(\log n)$ .

## Chapter 3

# Core Data Structures

### 3.1 Arrays

#### Definition 3.1.1: Array

A contiguous block of memory storing elements of the same type. Supports  $O(1)$  random access via index arithmetic. Python's `list` is a *dynamic array*.

| Operation                | Description             | Static | Dynamic  |
|--------------------------|-------------------------|--------|----------|
| Access <code>a[i]</code> | Read/write by index     | $O(1)$ | $O(1)$   |
| Search                   | Find element (unsorted) | $O(n)$ | $O(n)$   |
| Insert at end            | Append                  | N/A    | $O(1)^*$ |
| Insert at index $i$      | Shift elements right    | $O(n)$ | $O(n)$   |
| Delete at end            | Pop                     | N/A    | $O(1)^*$ |
| Delete at index $i$      | Shift elements left     | $O(n)$ | $O(n)$   |

#### Remark.

\* Amortized. A dynamic array doubles its capacity when full, costing  $O(n)$  occasionally, but the amortized cost per append is  $O(1)$ .

### 3.2 Stacks

#### Definition 3.2.1: Stack

A LIFO (Last-In, First-Out) abstract data type supporting: **push** (add to top), **pop** (remove from top), **peek** (read top). All operations in  $O(1)$ . Implemented with a list in Python.

| Operation | Python                   | Time     |
|-----------|--------------------------|----------|
| Push      | <code>s.append(x)</code> | $O(1)^*$ |
| Pop       | <code>s.pop()</code>     | $O(1)^*$ |
| Peek      | <code>s[-1]</code>       | $O(1)$   |
| Is empty  | <code>len(s) == 0</code> | $O(1)$   |

**Remark.**

- Classic applications: function call stack, expression parsing (balanced parentheses), DFS, undo/redo, monotonic stack problems.

### 3.3 Queues

#### Definition 3.3.1: Queue

A FIFO (First-In, First-Out) ADT supporting: **enqueue** (add to back), **dequeue** (remove from front), **peek**. Use `collections.deque` in Python —  $O(1)$  both ends. Using a plain `list` gives  $O(n)$  dequeue.

| Operation  | Python (deque)               | Time   |
|------------|------------------------------|--------|
| Enqueue    | <code>q.append(x)</code>     | $O(1)$ |
| Dequeue    | <code>q.popleft()</code>     | $O(1)$ |
| Peek front | <code>q[0]</code>            | $O(1)$ |
| Peek back  | <code>q[-1]</code>           | $O(1)$ |
| Push left  | <code>q.appendleft(x)</code> | $O(1)$ |
| Pop right  | <code>q.pop()</code>         | $O(1)$ |

**Remark.**

Applications: BFS, task scheduling, sliding window problems (monotonic deque). `collections.deque` is a doubly-linked list under the hood.

### 3.4 Linked Lists

#### Definition 3.4.1: Linked List

A sequence of *nodes*, each storing a value and a pointer to the next node. No random access; traversal is  $O(n)$ .

| Operation          | Description                | Singly         | Doubly |
|--------------------|----------------------------|----------------|--------|
| Access by index    | Traverse from head         | $O(n)$         | $O(n)$ |
| Search             | Linear scan                | $O(n)$         | $O(n)$ |
| Insert at head     | Update head pointer        | $O(1)$         | $O(1)$ |
| Insert at tail     | Requires tail pointer      | $O(1)^*$       | $O(1)$ |
| Insert at position | Traverse then insert       | $O(n)$         | $O(n)$ |
| Delete at head     | Update head pointer        | $O(1)$         | $O(1)$ |
| Delete at tail     | Traverse to second-to-last | $O(n)$         | $O(1)$ |
| Delete given node  | Unlink from neighbours     | $O(n)^\dagger$ | $O(1)$ |

**Remark.**

\* Requires a maintained tail pointer.  $^\dagger$  Need to find predecessor; in a doubly linked list, given the node, deletion is  $O(1)$ .

| Feature              | Array      | Linked List                       |
|----------------------|------------|-----------------------------------|
| Access by index      | $O(1)$     | $O(n)$                            |
| Insert/delete head   | $O(n)$     | $O(1)$                            |
| Insert/delete middle | $O(n)$     | $O(n)$ search + $O(1)$            |
| Memory               | Contiguous | Non-contiguous (pointer overhead) |
| Cache friendly       | Yes        | No                                |

### 3.5 Trees

#### Definition 3.5.1: Binary Tree

A rooted tree where each node has at most two children (left and right). Height  $h$  of a tree is the length of the longest root-to-leaf path. A *complete* binary tree has all levels full except possibly the last. A *full* binary tree has every node with 0 or 2 children.

#### Definition 3.5.2: Binary Search Tree (BST)

A binary tree where for every node  $v$ :

- all keys in the *left* subtree are **less than**  $v.key$
- all keys in the *right* subtree are **greater than**  $v.key$

Average case:  $O(\log n)$  for search, insert, delete. Worst case (skewed):  $O(n)$ .

#### Tree Traversals

| Traversal   | Order                           | Visit pattern     | Common use                |
|-------------|---------------------------------|-------------------|---------------------------|
| Inorder     | $L \rightarrow N \rightarrow R$ | Left, root, right | BST sorted output         |
| Preorder    | $N \rightarrow L \rightarrow R$ | Root, left, right | Copy tree, prefix expr    |
| Postorder   | $L \rightarrow R \rightarrow N$ | Left, right, root | Delete tree, postfix expr |
| Level-order | by depth                        | BFS row by row    | Level-by-level processing |

#### BST Operations

| Operation | Average     | Worst (skewed) |
|-----------|-------------|----------------|
| Search    | $O(\log n)$ | $O(n)$         |
| Insert    | $O(\log n)$ | $O(n)$         |
| Delete    | $O(\log n)$ | $O(n)$         |
| Min / Max | $O(\log n)$ | $O(n)$         |

#### Remark.

Self-balancing BSTs (AVL, Red-Black) guarantee  $O(\log n)$  worst case. Python's `heapq` implements a min-heap (complete binary tree stored as array).

# Chapter 4

## Sorting

### 4.1 Comparison-Based Sorting Algorithms

#### Insertion Sort

**Definition 4.1.1: Insertion Sort**

Build a sorted prefix one element at a time. For each element, shift it leftward into its correct position in the sorted portion.

---

**Algorithm 1** Insertion Sort

---

```
1: for  $i = 1$  to  $n - 1$  do
2:    $key \leftarrow A[i]$ 
3:    $j \leftarrow i - 1$ 
4:   while  $j \geq 0$  and  $A[j] > key$  do
5:      $A[j + 1] \leftarrow A[j]$ 
6:      $j \leftarrow j - 1$ 
7:   end while
8:    $A[j + 1] \leftarrow key$ 
9: end for
```

---

#### Merge Sort

**Definition 4.1.2: Merge Sort**

Divide array in half, recursively sort each half, merge the two sorted halves. Recurrence:  $T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$ .

**Algorithm 2** Merge Sort

---

```

1: procedure MERGESORT( $A, l, r$ )
2:   if  $l < r$  then
3:      $m \leftarrow \lfloor (l + r)/2 \rfloor$ 
4:     MERGESORT( $A, l, m$ )
5:     MERGESORT( $A, m + 1, r$ )
6:     MERGE( $A, l, m, r$ )
7:   end if
8: end procedure

```

---

**Remark.**

**Merge** takes two sorted subarrays and combines them in  $O(n)$  by comparing elements front-to-back. Requires  $O(n)$  auxiliary space.

**Quick Sort****Definition 4.1.3: Quick Sort**

Choose a *pivot*, partition the array so all elements  $\leq$  pivot are to its left and all  $>$  pivot to its right, then recurse on each side.

**Algorithm 3** Quick Sort

---

```

1: procedure QUICKSORT( $A, l, r$ )
2:   if  $l < r$  then
3:      $p \leftarrow \text{PARTITION}(A, l, r)$ 
4:     QUICKSORT( $A, l, p - 1$ )
5:     QUICKSORT( $A, p + 1, r$ )
6:   end if
7: end procedure

```

---

**Remark.**

Partition (Lomuto scheme): pick last element as pivot; maintain index of “smaller” region. Average  $O(n \log n)$ ; worst case (already sorted)  $O(n^2)$ . Randomised pivot makes worst case extremely unlikely.

**Heap Sort****Definition 4.1.4: Heap Sort**

Build a max-heap from the array ( $O(n)$ ), then repeatedly extract the maximum and place it at the end ( $O(n \log n)$ ). Total:  $O(n \log n)$ , in-place, *not* stable.

**Selection Sort**

Find the minimum of the unsorted portion and swap it to the front. Always  $O(n^2)$ , regardless of input. *Not* stable.

## 4.2 Lower Bound for Comparison Sorting

### Theorem 4.2.1: $\Omega(n \log n)$ Lower Bound

Any comparison-based sorting algorithm requires  $\Omega(n \log n)$  comparisons in the worst case.

#### Proof

Model any comparison sort as a *decision tree*: each internal node is a comparison  $A[i] \leq A[j]$  (yes/no), and each leaf is a permutation of the sorted output. A correct algorithm must be able to produce all  $n!$  orderings, so the decision tree has  $\geq n!$  leaves. A binary tree of height  $h$  has at most  $2^h$  leaves, so  $2^h \geq n!$ , giving  $h \geq \log_2(n!) = \Omega(n \log n)$  (by Stirling's approximation).  $\square$

## 4.3 Sorting Algorithms Summary

| Algorithm      | Best          | Average       | Worst         | Space       | Stable |
|----------------|---------------|---------------|---------------|-------------|--------|
| Insertion Sort | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$      | Yes    |
| Selection Sort | $O(n^2)$      | $O(n^2)$      | $O(n^2)$      | $O(1)$      | No     |
| Merge Sort     | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(n)$      | Yes    |
| Quick Sort     | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$      | $O(\log n)$ | No     |
| Heap Sort      | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(1)$      | No     |
| Counting Sort  | $O(n + k)$    | $O(n + k)$    | $O(n + k)$    | $O(k)$      | Yes    |
| Radix Sort     | $O(d(n + k))$ | $O(d(n + k))$ | $O(d(n + k))$ | $O(n + k)$  | Yes    |

#### Remark.

Counting and Radix Sort are *not* comparison-based; they beat the  $\Omega(n \log n)$  lower bound by exploiting key structure.  $k$  = key range,  $d$  = number of digits.



# Chapter 5

## Graph Algorithms

### 5.1 Graph Basics

#### Definition 5.1.1: Graph

$G = (V, E)$  where  $V$  is a set of vertices and  $E \subseteq V \times V$  is a set of edges. A graph is *directed* (digraph) if edges have orientation, *undirected* otherwise.  $n = |V|$ ,  $m = |E|$ .

| Representation   | Description                                                 | Space      | Edge check  |
|------------------|-------------------------------------------------------------|------------|-------------|
| Adjacency Matrix | $n \times n$ boolean matrix; $M[u][v]=1$ if edge exists     | $O(n^2)$   | $O(1)$      |
| Adjacency List   | Array of lists; $\text{adj}[u]$ = list of neighbours of $u$ | $O(n + m)$ | $O(\deg u)$ |

#### Remark.

Use adjacency lists for sparse graphs ( $m \ll n^2$ ), adjacency matrix for dense graphs or when  $O(1)$  edge queries are required.

### 5.2 Breadth-First Search (BFS)

#### Definition 5.2.1: BFS

Explore all vertices at distance  $d$  from source before distance  $d + 1$ . Uses a queue. Computes shortest-path distances (unweighted) from source.

**Algorithm 4** BFS

---

```

1: procedure BFS( $G, s$ )
2:   Mark all vertices unvisited;  $\text{dist}[s] \leftarrow 0$ 
3:    $Q \leftarrow$  empty queue; enqueue  $s$ ; mark  $s$  visited
4:   while  $Q$  not empty do
5:      $u \leftarrow Q.\text{DEQUEUE}()$ 
6:     for each neighbour  $v$  of  $u$  do
7:       if  $v$  not visited then
8:         mark  $v$  visited;  $\text{dist}[v] \leftarrow \text{dist}[u] + 1$ 
9:          $Q.\text{ENQUEUE}(v)$ 
10:      end if
11:    end for
12:  end while
13: end procedure

```

---

Time:  $O(n + m)$ . Space:  $O(n)$  for queue and visited array.

### 5.3 Depth-First Search (DFS) & Applications

#### Definition 5.3.1: DFS

Explore as deep as possible before backtracking. Uses a stack (or recursion). Produces DFS tree with *tree*, *back*, *forward*, and *cross* edges (directed graphs).

**Algorithm 5** DFS

---

```

1: procedure DFS-VISIT( $G, u$ )
2:   mark  $u$  visited; push  $u$  onto stack (for topological order)
3:   for each neighbour  $v$  of  $u$  do
4:     if  $v$  not visited then
5:       DFS-VISIT( $G, v$ )
6:     end if
7:   end for
8:   record finish time of  $u$ 
9: end procedure

```

---

Time:  $O(n + m)$ .

---

| Edge type    | Meaning                                                          |
|--------------|------------------------------------------------------------------|
| Tree edge    | $v$ first discovered via $u$                                     |
| Back edge    | $v$ is an ancestor of $u$ (indicates a cycle in directed graphs) |
| Forward edge | $v$ is a descendant of $u$ but not through tree edge             |
| Cross edge   | $v$ is in a different subtree                                    |

---

## 5.4 Topological Sort

### Definition 5.4.1: Topological Sort

A linear ordering of vertices of a *DAG* (directed acyclic graph) such that for every edge  $(u, v)$ ,  $u$  appears before  $v$ .

**DFS-based algorithm:** Run DFS; output vertices in decreasing finish time.  $O(n + m)$ .

**Kahn's algorithm (BFS-based):** Repeatedly remove vertices with in-degree 0.  $O(n + m)$ .

**Remark.**

A directed graph has a topological sort *if and only if* it is a DAG (no back edges in DFS).

## 5.5 Strongly Connected Components (SCC)

### Definition 5.5.1: SCC

A maximal set of vertices  $C \subseteq V$  such that every vertex in  $C$  is reachable from every other vertex in  $C$ .

**Kosaraju's Algorithm** ( $O(n + m)$ ):

1. Run DFS on  $G$ ; record finish times.
2. Compute  $G^T$  (transpose: reverse all edges).
3. Run DFS on  $G^T$  in decreasing finish-time order; each DFS tree is one SCC.

**Tarjan's Algorithm** ( $O(n + m)$ ): Single DFS with a stack; uses *discovery times* and *low-link values* to identify SCCs.

## 5.6 Minimum Spanning Trees (MST)

### Definition 5.6.1: Minimum Spanning Tree

Given a connected, undirected, weighted graph  $G = (V, E, w)$ , an MST is a spanning tree  $T \subseteq E$  that minimises  $\sum_{e \in T} w(e)$ . Any spanning tree has exactly  $n - 1$  edges.

### Kruskal's Algorithm

Sort all edges by weight; add each edge if it doesn't form a cycle (use Union-Find).

**Algorithm 6** Kruskal's MST

---

```

1: Sort edges:  $e_1 \leq e_2 \leq \dots \leq e_m$ 
2:  $T \leftarrow \emptyset$ ; initialise Union-Find on  $V$ 
3: for each edge  $(u, v)$  in sorted order do
4:   if FIND( $u$ )  $\neq$  FIND( $v$ ) then
5:      $T \leftarrow T \cup \{(u, v)\}$ 
6:     UNION( $u, v$ )
7:   end if
8: end for
9: return  $T$ 

```

---

Time:  $O(m \log m)$  dominated by sorting (or  $O(m\alpha(n))$  with path compression).

**Prim's Algorithm**

Grow the MST greedily from a starting vertex; always add the minimum-weight edge crossing the cut between tree and non-tree vertices.

Time:  $O(m \log n)$  with a binary heap;  $O(m + n \log n)$  with a Fibonacci heap.

**5.7 Shortest Paths****Single-Source Shortest Paths****Definition 5.7.1: Dijkstra's Algorithm**

Greedy SSSP for graphs with *non-negative* edge weights. Maintains a priority queue of (distance, vertex) pairs.  
Time:  $O((n + m) \log n)$  with a binary heap.

**Algorithm 7** Dijkstra's Algorithm

---

```

1:  $\text{dist}[s] \leftarrow 0$ ;  $\text{dist}[v] \leftarrow \infty$  for all  $v \neq s$ 
2: Priority queue  $PQ \leftarrow \{(0, s)\}$ 
3: while  $PQ$  not empty do
4:    $(d, u) \leftarrow PQ.\text{EXTRACTMIN}()$ 
5:   if  $d > \text{dist}[u]$  then continue
6:   end if
7:   for each edge  $(u, v, w)$  do
8:     if  $\text{dist}[u] + w < \text{dist}[v]$  then
9:        $\text{dist}[v] \leftarrow \text{dist}[u] + w$ 
10:       $PQ.\text{INSERT}((\text{dist}[v], v))$ 
11:     end if
12:   end for
13: end while

```

---

**Definition 5.7.2: Bellman-Ford**

SSSP for graphs with *negative* edge weights (no negative cycles). Relax all  $m$  edges  $n - 1$  times. Time:  $O(nm)$ .  
Detects negative cycles: if any edge can be relaxed after  $n - 1$  rounds, a negative cycle is reachable from  $s$ .

## All-Pairs Shortest Paths

**Definition 5.7.3: Floyd-Warshall**

Computes shortest paths between *all* pairs of vertices. DP approach:  $d^{(k)}[i][j]$  = shortest path from  $i$  to  $j$  using only vertices  $\{1, \dots, k\}$  as intermediates.

$$d^{(k)}[i][j] = \min\left(d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j]\right)$$

Time:  $O(n^3)$ . Space:  $O(n^2)$ .

| Algorithm      | Scope | Weights            | Time                | Space    |
|----------------|-------|--------------------|---------------------|----------|
| BFS            | SSSP  | Unweighted         | $O(n + m)$          | $O(n)$   |
| Dijkstra       | SSSP  | $\geq 0$           | $O((n + m) \log n)$ | $O(n)$   |
| Bellman-Ford   | SSSP  | Any (no neg cycle) | $O(nm)$             | $O(n)$   |
| Floyd-Warshall | APSP  | Any (no neg cycle) | $O(n^3)$            | $O(n^2)$ |

## Chapter 6

# Algorithm Design Paradigms

### 6.1 Divide & Conquer

#### Definition 6.1.1: Divide & Conquer

Recursively break a problem into  $a$  subproblems of size  $n/b$ , solve each, then combine. Cost:  $T(n) = aT(n/b) + f(n)$ .

#### Closest Pair of Points

**Problem:** Given  $n$  points in the plane, find the pair with minimum Euclidean distance. Naive brute-force:  $O(n^2)$ .

**Algorithm** ( $O(n \log n)$ ):

1. Sort points by  $x$ -coordinate.
2. Divide at median  $x$ ; recursively find  $\delta_L$  and  $\delta_R$ .
3. Let  $\delta = \min(\delta_L, \delta_R)$ .
4. **Combine:** Consider the vertical strip of width  $2\delta$  around the dividing line. For each point in the strip (sorted by  $y$ ), check at most **7** subsequent points — the geometry bounds the density.
5. Return  $\min(\delta, \delta_{\text{strip}})$ .

#### Theorem 6.1.2: Closest Pair Complexity

$T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$  by Master Theorem Case 2.

### 6.2 Greedy Algorithms

#### Definition 6.2.1: Greedy Algorithm

Make the locally optimal choice at each step, hoping to reach a global optimum. Correctness requires an *exchange argument* or *greedy stays ahead* proof.

## Interval Scheduling

**Problem:** Given  $n$  intervals  $[s_i, f_i]$ , select the maximum number of non-overlapping intervals.

**Algorithm:** Sort by finish time  $f_i$ ; greedily add intervals that start after the last selected one finishes.

### Theorem 6.2.2: Interval Scheduling Greedy is Optimal

**Proof (Exchange argument):** Let  $G = \{g_1, g_2, \dots, g_k\}$  be greedy solution,  $O = \{o_1, o_2, \dots, o_m\}$  an optimal solution, both sorted by finish time. Show by induction:  $f(g_i) \leq f(o_i)$  for all  $i$ . Since greedy finishes no later than optimal at each step, it can always fit at least as many intervals. Hence  $k \geq m$ , so  $k = m$ .

## Huffman Coding

**Problem:** Build an optimal prefix-free binary code minimising total encoded length, given symbol frequencies.

**Algorithm:** Use a min-heap; repeatedly merge the two nodes with lowest frequency into a new node with their combined frequency.

Time:  $O(n \log n)$ .

**Remark.**

■ Huffman coding is provably optimal for prefix-free codes.

## 6.3 Dynamic Programming

### Definition 6.3.1: Dynamic Programming

Solve a problem by breaking it into overlapping subproblems, storing results (*memoisation* or bottom-up table), and combining. Applicable when the problem has: (1) **Optimal substructure:** optimal solution contains optimal sub-solutions. (2) **Overlapping subproblems:** subproblems recur.

### Longest Common Subsequence (LCS)

**Problem:** Given strings  $X[1..m]$  and  $Y[1..n]$ , find the longest subsequence common to both.

**Recurrence:**

$$\text{dp}[i][j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ \text{dp}[i-1][j-1] + 1 & \text{if } X[i] = Y[j] \\ \max(\text{dp}[i-1][j], \text{dp}[i][j-1]) & \text{otherwise} \end{cases}$$

Time:  $O(mn)$ . Space:  $O(mn)$  (reducible to  $O(\min(m, n))$ ).

### 0/1 Knapsack

**Problem:**  $n$  items each with weight  $w_i$  and value  $v_i$ ; knapsack capacity  $W$ . Maximise total value with total weight  $\leq W$ .

**Recurrence:**

$$\text{dp}[i][w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ \text{dp}[i-1][w] & \text{if } w_i > w \\ \max(\text{dp}[i-1][w], v_i + \text{dp}[i-1][w - w_i]) & \text{otherwise} \end{cases}$$

Time:  $O(nW)$ . Space:  $O(nW)$  (reducible to  $O(W)$ ).

**Remark.**

- This is *pseudo-polynomial*: exponential in the input size (bits of  $W$ ) but polynomial in the *value*  $W$ .

**Weighted Interval Scheduling (Interval DP)**

**Problem:** Select non-overlapping intervals to maximise total weight.

Sort intervals by finish time. Let  $p(i) = \text{largest } j < i \text{ with } f_j \leq s_i$ .

$$\text{dp}[i] = \max(\text{dp}[i-1], w_i + \text{dp}[p(i)])$$

Time:  $O(n \log n)$  (binary search for  $p(i)$ ).

**6.4 Matching****Bipartite Matching****Definition 6.4.1: Bipartite Graph**

$G = (U \cup V, E)$  where all edges go between  $U$  and  $V$ . A *matching*  $M \subseteq E$  is a set of edges with no shared endpoints. A *maximum matching* maximises  $|M|$ .

**Algorithm** (Augmenting Paths, Hopcroft-Karp): Find shortest augmenting paths (paths from unmatched  $U$  to unmatched  $V$  alternating matched/unmatched edges). Time:  $O(\sqrt{n} \cdot m)$ .

**Stable Matching (Gale-Shapley)**

**Problem:** Given  $n$  men and  $n$  women each with preference rankings, find a stable matching (no blocking pair: a man-woman pair who both prefer each other over current partners).

**Algorithm 8 Gale-Shapley (Propose-and-Reject)**


---

```

1: while any man is unmatched do
2:    $m$  proposes to his highest-ranked woman  $w$  not yet proposed to
3:   if  $w$  is unmatched then
4:     tentatively match  $(m, w)$ 
5:   else if  $w$  prefers  $m$  over current partner  $m'$  then
6:     unmatched  $m'$ ; tentatively match  $(m, w)$ 
7:   end if
8: end while

```

---

**Theorem 6.4.2: Gale-Shapley Correctness**

The algorithm terminates in  $O(n^2)$  and produces a *stable* matching. The result is **man-optimal** (each man gets his best stable partner) and **woman-pessimal** (each woman gets her worst stable partner).



# Chapter 7

## Computability & Complexity

### 7.1 Computability

#### Definition 7.1.1: Turing Machine

A mathematical model of computation consisting of an infinite tape, a read/write head, a finite set of states, and a transition function. Defines what is *computable* in principle.

#### Definition 7.1.2: Decidable Language

A language  $L$  is *decidable* (recursive) if there exists a Turing machine that *always halts* and correctly accepts  $x \in L$  and rejects  $x \notin L$ .

#### Definition 7.1.3: Recognisable Language

A language  $L$  is *recognisable* (recursively enumerable) if there exists a Turing machine that accepts  $x \in L$  (but may loop on  $x \notin L$ ).

#### Theorem 7.1.4: Halting Problem is Undecidable

Let  $\text{HALT} = \{\langle M, x \rangle \mid M \text{ halts on input } x\}$ .  $\text{HALT}$  is not decidable.

#### Proof sketch

Assume for contradiction a decider  $H$  for  $\text{HALT}$  exists. Construct  $D$ : on input  $\langle M \rangle$ , run  $H(\langle M, \langle M \rangle \rangle)$ ; if  $H$  accepts (i.e.  $M$  halts on  $\langle M \rangle$ ), then  $D$  loops; if  $H$  rejects,  $D$  halts. Evaluate  $D$  on  $\langle D \rangle$ : contradiction in both cases. Hence  $H$  cannot exist.  $\square$

## 7.2 NP-Completeness

### Definition 7.2.1: Complexity Classes

- **P**: Decision problems solvable in polynomial time.
- **NP**: Decision problems with solutions *verifiable* in polynomial time. Equivalently, solvable by a nondeterministic TM in polynomial time.
- **NP-hard**: At least as hard as every problem in NP (via polynomial-time many-one reductions). Need not be in NP.
- **NP-complete**: In NP *and* NP-hard.

#### Remark.

$\mathbf{P} \subseteq \mathbf{NP}$ . Whether  $\mathbf{P} = \mathbf{NP}$  is the central open problem of computer science.

### Definition 7.2.2: Polynomial-Time Reduction

$A \leq_p B$  (“ $A$  reduces to  $B$ ”) if there exists a polynomial-time computable function  $f$  such that  $x \in A \iff f(x) \in B$ . If  $B \in P$  then  $A \in P$ ; contrapositively, if  $A \notin P$  then  $B \notin P$ .

### Theorem 7.2.3: Cook-Levin Theorem

3-SAT is NP-complete. (Every NP problem reduces to 3-SAT.)

## Classic NP-Complete Problems

| Problem          | Description                                                     |
|------------------|-----------------------------------------------------------------|
| 3-SAT            | Boolean formula in 3-CNF; satisfying assignment?                |
| INDEPENDENT SET  | Graph $G$ , integer $k$ ; independent set of size $\geq k$ ?    |
| CLIQUE           | Graph $G$ , integer $k$ ; clique of size $\geq k$ ?             |
| VERTEX COVER     | Graph $G$ , integer $k$ ; vertex cover of size $\leq k$ ?       |
| 3-COLORING       | Can vertices of $G$ be colored with 3 colors, no adjacent same? |
| HAMILTONIAN PATH | Does $G$ have a path visiting every vertex exactly once?        |
| TSP (DECISION)   | Tour visiting all cities with total cost $\leq k$ ?             |
| SUBSET SUM       | Integers $S$ , target $t$ ; subset summing to exactly $t$ ?     |

## Key Reductions

### 3-SAT $\leq_p$ Independent Set

**Construction:** Given a 3-CNF formula with  $k$  clauses, build graph  $G$ :

- For each clause, create a *triangle* (3 nodes, one per literal).
- Add an edge between any two nodes in different triangles representing *complementary* literals ( $x$  and  $\bar{x}$ ).

### Theorem 7.2.4: Reduction correctness

$\phi$  is satisfiable  $\iff G$  has an independent set of size  $k$ .

- $(\Rightarrow)$  A satisfying assignment selects one true literal per clause; these  $k$  nodes form an independent set (no edges within a triangle since we pick one; no inter-triangle edges since complementary literals can't both be true).
- $(\Leftarrow)$  An independent set of size  $k$  picks exactly one node per triangle; assign those literals true (no contradiction since complement pairs are excluded).

### Independent Set $\leq_p$ Vertex Cover

### Theorem 7.2.5: Complement duality

$S$  is an independent set of size  $k$  in  $G \iff V \setminus S$  is a vertex cover of size  $n - k$ .

#### Proof

$(\Rightarrow)$  For any edge  $(u, v)$ : since  $S$  is independent, not both  $u, v \in S$ , so at least one of  $u, v \in V \setminus S$ . Hence  $V \setminus S$  covers every edge.

$(\Leftarrow)$  Suppose  $u, v \in S$ . Then neither is in  $V \setminus S$ , but edge  $(u, v)$  is uncovered — contradiction. So  $S$  is independent.  $\square$

### 3-SAT $\leq_p$ 3-Coloring

**Construction:** Build a graph with a *palette* gadget (3 nodes: T, F, B pairwise connected) and a *clause gadget* for each clause that forces at least one literal to match T. The reduction preserves satisfiability.

### Coping with NP-Completeness

| Strategy                  | Description                                                                              |
|---------------------------|------------------------------------------------------------------------------------------|
| Special cases             | Some NP-complete problems are poly-time on restricted inputs (e.g. 2-SAT, planar graphs) |
| Approximation             | Guarantee solution within factor $\alpha$ of optimal                                     |
| FPT algorithms            | Exponential only in a small parameter $k$ (e.g. $O(2^k \cdot n)$ )                       |
| Heuristics / Local search | No guarantee, but fast in practice                                                       |

Part III

Computer Systems

# Chapter 8

## Digital Foundations

### 8.1 Binary & Number Systems

#### Definition 8.1.1: Bit and Byte

A **bit** (binary digit) is the smallest unit of information, taking value 0 or 1. Eight bits form one **byte**. Common multiples:  $1 \text{ KB} = 2^{10} \text{ bytes}$ ,  $1 \text{ MB} = 2^{20} \text{ bytes}$ ,  $1 \text{ GB} = 2^{30} \text{ bytes}$ .

Computers represent all data—numbers, text, images, programs—as sequences of bits. A group of  $n$  bits can encode  $2^n$  distinct values.

#### Positional Notation

Every integer can be written in any base. The **binary** (base-2) and **hexadecimal** (base-16) systems are most common in computing.

| Base             | Example: the number 42                                                                         |
|------------------|------------------------------------------------------------------------------------------------|
| Decimal (10)     | 42                                                                                             |
| Binary (2)       | $101010_2 = 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0$ |
| Hexadecimal (16) | $2A_{16} = 2 \cdot 16^1 + 10 \cdot 16^0$                                                       |

#### Remark.

Hex is a compact shorthand for binary: each hex digit maps to exactly four bits ( $0 \rightarrow 0000, \dots, F \rightarrow 1111$ ). Memory addresses and color codes ( $\#FF5733$ ) are almost always written in hex.

#### Integer Encoding

Modern machines use 32-bit or 64-bit integers. A 32-bit *unsigned* integer can represent 0 to  $2^{32} - 1 \approx 4.3 \times 10^9$ . *Signed* integers use **two's complement**: flip all bits and add 1 to negate; the most significant bit is a sign bit.

#### Example 8.1.1

In 8-bit two's complement:  $+5 = 00000101$ ,  $-5 = 11111011$ . Range:  $-128$  to  $127$ .

# 8.2 Boolean Algebra & Logic Gates

## Historical Context

| Who                    | Contribution                                                                     |
|------------------------|----------------------------------------------------------------------------------|
| Aristotle (~300 BC)    | Formal logic — rules for valid reasoning                                         |
| G. Leibniz (~1670s)    | Stepped Reckoner; envisioned mechanical calculation                              |
| G. Boole (1854)        | <i>An Investigation of the Laws of Thought</i> — algebra on True/False           |
| C. Shannon (1938)      | Showed Boolean algebra maps one-to-one to electrical circuits; defined the “bit” |
| A. Turing (1937)       | Turing machine — theoretical model of computation                                |
| J. von Neumann (~1945) | Physical computer architecture; ENIAC (1945), Mac/PC (1980s)                     |

### Definition 8.2.1: Boolean Variable

A variable that can only be **True** (1) or **False** (0). Boolean *operators* combine Boolean variables into expressions.

## Basic Operators and Truth Tables

Three *primitive* operators suffice to express any Boolean function:

| Operator | Symbol         | Rule                   |
|----------|----------------|------------------------|
| NOT      | $\overline{A}$ | Flip the bit           |
| AND      | $A \cdot B$    | 1 only if both are 1   |
| OR       | $A + B$        | 1 if at least one is 1 |

Derived operators frequently appear in hardware:

| Gate | Expression             | Outputs 1 when...              |
|------|------------------------|--------------------------------|
| XOR  | $A \oplus B$           | inputs differ                  |
| NAND | $\overline{A \cdot B}$ | NOT (A AND B) — universal gate |
| NOR  | $\overline{A + B}$     | NOT (A OR B) — universal gate  |

### Remark.

NAND and NOR are each *universal*: any Boolean function can be built from NAND gates alone (or NOR gates alone). Modern chips are designed almost entirely with NAND gates.

### Example 8.2.1

Truth table for XOR ( $A \oplus B$ ):

| A | B | $A \oplus B$ |
|---|---|--------------|
| 0 | 0 | 0            |
| 0 | 1 | 1            |
| 1 | 0 | 1            |
| 1 | 1 | 0            |

# From Algebra to Transistors

Shannon’s 1938 insight: a high-voltage electrical signal = 1, low voltage = 0. Transistors act as switches: a *control* voltage opens or closes the path for current, implementing NOT, AND, OR in silicon. Modern CPUs contain billions of transistors.

## 8.3 Basic Circuits

### Combinational Circuits

Circuits whose output depends only on *current* inputs (no memory).

| Circuit           | Function                                                                      |
|-------------------|-------------------------------------------------------------------------------|
| Half adder        | Adds two bits; outputs $\text{Sum} = A \oplus B$ , $\text{Carry} = A \cdot B$ |
| Full adder        | Adds two bits plus a carry-in; chains to build $n$ -bit adders                |
| Multiplexer (MUX) | Selector: outputs one of $n$ inputs based on a control signal                 |
| Decoder           | Maps $N$ inputs to $2^N$ outputs; exactly one output is active (lookup table) |

### Sequential Circuits and Memory

Sequential circuits *store state*: output depends on input *and* past state.

#### Definition 8.3.1: Flip-Flop

A circuit built from two D-latches sharing a clock signal (CLK). On each rising clock edge it captures its input  $D$  and holds the value until the next edge. A **register** is a bank of flip-flops sharing one CLK, storing a multi-bit value (e.g., a 64-bit register holds one machine word).

#### Remark.

The clock signal oscillates between 0 and 1 at a fixed frequency (e.g.,  $4\text{GHz} = 4 \times 10^9$  cycles/second), giving every circuit component a shared sense of “time.” A higher clock speed means more operations per second, but also more heat and power consumption.

## Chapter 9

# Computer Organization

### 9.1 A Brief History of Computing

Two independent intellectual threads merged in the 1940s to create the modern computer:

**Thread 1 — Logic & Circuits** (Aristotle → Leibniz → Boole → Shannon): showed that logical reasoning can be reduced to mathematical operations and that those operations can be performed by electrical circuits.

**Thread 2 — Theory of Computation** (Hilbert → Turing): asked whether mathematics is decidable. Turing’s answer (1937) was “not entirely,” and his proof introduced the **Turing machine** — the theoretical blueprint for any computer.

#### Definition 9.1.1: Turing Machine

An abstract device with:

- an infinitely long **tape** divided into cells (each stores a symbol);
- a **read/write head** that can read the current cell, write a new symbol, and move one step left or right;
- a **state table** (program) mapping (current state, symbol read) → (new state, symbol to write, direction to move).

Despite its simplicity, a Turing machine can compute anything computable.

#### Remark.

**Turing completeness:** a system is Turing-complete if it can simulate a Turing machine. Python is Turing-complete (a list acts as the tape; the interpreter is the head). The **halting problem** (can a program predict whether any given program will halt?) is *undecidable* — Turing’s most important result.



# 9.2 The Von Neumann Model

## Definition 9.2.1: Von Neumann Architecture

A physical computer that realises the Turing machine using electrical circuits. The mapping:

Tape  $\rightarrow$  Memory      Head  $\rightarrow$  CPU      State table  $\rightarrow$  Programs (stored in memory)

Key principle: **stored-program** — code and data share the same memory.

| Component                   | Role                                                               |
|-----------------------------|--------------------------------------------------------------------|
| Control Unit (CU)           | Fetches and decodes instructions; coordinates all other units      |
| Arithmetic/Logic Unit (ALU) | Executes arithmetic (+, −, ×, ÷) and logic (AND, OR, NOT, compare) |
| Registers                   | Tiny, fastest storage inside the CPU; hold operands and results    |
| Memory (RAM)                | Stores both the running program and its data                       |
| Input / Output              | Keyboard, display, network interface, storage devices              |

# 9.3 Registers, ALU, and the Instruction Cycle

## Registers

Registers are the CPU’s working scratchpad — typically 16 to 32 general-purpose registers, each 64 bits wide on modern processors. Special registers include the **Program Counter (PC)**, which holds the address of the *next* instruction to execute, and the **Stack Pointer (SP)**.

## The Instruction Cycle

Every program reduces to a repeated three-step cycle:

| Stage          | What happens                                                     |
|----------------|------------------------------------------------------------------|
| <b>Fetch</b>   | CU reads the instruction at address PC from memory; PC += 1      |
| <b>Decode</b>  | CU interprets the instruction’s opcode and operands              |
| <b>Execute</b> | ALU performs the operation; result written to register or memory |

### Remark.

Modern CPUs execute millions of instructions per second by *pipelining* (overlapping fetch/decode/execute for different instructions) and running multiple cores in parallel. A 3.5 GHz quad-core chip can complete tens of billions of simple operations per second.

# Chapter 10

## Memory Hierarchy

### 10.1 Memory Types

#### Primary Memory (directly CPU-accessible)

| Type                       | Volatile? | Notes                                                 |
|----------------------------|-----------|-------------------------------------------------------|
| RAM (Random Access Memory) | Yes       | Working memory; OS and programs load here at run time |
| ROM (Read-Only Memory)     | No        | Stores firmware (BIOS/UEFI); survives power-off       |

#### Secondary Memory (not CPU-accessible directly)

| Type                    | Technology        | Characteristics                                                 |
|-------------------------|-------------------|-----------------------------------------------------------------|
| HDD (Hard Disk Drive)   | Magnetic platters | Cheap, high capacity ( $\sim$ TB), slow ( $\sim$ 10 ms seek)    |
| SSD (Solid State Drive) | NAND flash memory | Much faster than HDD, higher cost per GB, no moving parts       |
| USB / Flash drive       | NAND flash        | Portable; slower than SSD; not reliable for long-term archiving |

|             | RAM           | Hard Disk                                |
|-------------|---------------|------------------------------------------|
| Capacity    | $\sim$ GB     | $\sim$ TB ( $1000\times$ larger)         |
| Cost        | $\sim$ \\$/GB | $\sim$ \\$0.04/GB ( $200\times$ cheaper) |
| Access time | $\sim$ 100 ns | $\sim$ 10 ms ( $100,000\times$ slower)   |
| Volatility  | Volatile      | Non-volatile                             |

### 10.2 The Memory Hierarchy and Cache

#### Definition 10.2.1: Memory Hierarchy

Storage is organised in layers trading off *speed*, *cost*, and *capacity*. Closer to the CPU  $\Rightarrow$  faster & smaller & more expensive.

| Level              | Size        | Latency      | Managed by     |
|--------------------|-------------|--------------|----------------|
| CPU Registers      | < 1 KB      | ~1 cycle     | Compiler / CPU |
| L1 Cache           | 32–64 KB    | ~4 cycles    | CPU hardware   |
| L2 Cache           | 256 KB–1 MB | ~10 cycles   | CPU hardware   |
| L3 Cache (shared)  | 4–32 MB     | 40–75 cycles | CPU hardware   |
| Main Memory (DRAM) | 4–64 GB     | ~60 ns       | OS             |
| SSD                | 256 GB–2 TB | ~100 $\mu$ s | OS / FS        |
| HDD                | 1–10 TB     | ~10 ms       | OS / FS        |

**Remark.**

From 1980 to 2010, CPU speed improved roughly  $1000\times$  while DRAM speed improved only  $\sim 8\times$ . This *processor-memory gap* is why caches are indispensable.

### Locality Principles

Cache effectiveness relies on two empirical observations:

| Principle         | Meaning                                                                                          |
|-------------------|--------------------------------------------------------------------------------------------------|
| Temporal locality | A recently accessed address is likely to be accessed again soon (e.g., a loop variable)          |
| Spatial locality  | Addresses near a recently accessed address are likely to be accessed soon (e.g., array elements) |

When the CPU requests data:

- **Cache hit:** data is in the cache  $\rightarrow$  served in a few cycles.
- **Cache miss:** data must be fetched from the next level  $\rightarrow$  significant stall (potentially hundreds of cycles for DRAM).

### 10.3 Why This Matters for ML/DS

**Remark.**

Memory hierarchy is not just a hardware detail — it directly governs the speed of data-intensive workloads:

| Scenario          | Implication                                                                                                                                                                                         |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Loading a dataset | Reading from disk to RAM is the typical I/O bottleneck; SSDs help significantly; memory-mapped files avoid redundant copies                                                                         |
| NumPy / pandas    | Row-major (C) vs. column-major (Fortran) layout affects which access pattern gets cache-friendly sequential reads                                                                                   |
| Batch size (DL)   | Larger batches keep the GPU’s memory hierarchy busy; too small and overhead dominates; too large and you overflow GPU VRAM                                                                          |
| GPU memory        | GPUs have their own hierarchy: registers $\rightarrow$ shared memory $\rightarrow$ L1/L2 $\rightarrow$ global VRAM; optimising tensor layouts for spatial locality is the core of GPU kernel tuning |

## Part IV

# Computer Networking

# Chapter 11

## Network Fundamentals

### 11.1 The Internet and Packet Switching

#### Definition 11.1.1: The Internet

A global *network of networks*: billions of devices (end hosts) connected through a mesh of routers. No single entity owns or controls it.

#### Two Forwarding Paradigms

|              | Circuit Switching                                                              | Packet Switching                                                          |
|--------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| How it works | A dedicated path is reserved for the entire conversation (e.g., old telephone) | Data is split into independent <b>packets</b> ; each forwarded separately |
| Resources    | Reserved (guaranteed bandwidth)                                                | Shared (best-effort)                                                      |
| Waste        | Idle circuits waste capacity                                                   | No reservation; efficient                                                 |
| Used in      | PSTN telephony                                                                 | The Internet (IP)                                                         |

The Internet uses packet switching. Each packet carries a **destination IP address**; routers inspect that address and forward the packet toward its destination. Because resources are shared, packets can be *lost*, *delayed*, *reordered*, or *corrupted* — higher-layer protocols (TCP) deal with these problems.

### 11.2 Layered Architecture

#### Definition 11.2.1: Layering

Each layer provides a well-defined **service** to the layer above via an interface, hiding implementation details. This separation of concerns allows protocols to be changed without disturbing other layers.

#### TCP/IP Model

| Layer       | Protocols                 | Identifier              | Unit               |
|-------------|---------------------------|-------------------------|--------------------|
| Application | HTTP, DNS, SMTP, FTP, SSH | URL / hostname          | Message            |
| Transport   | TCP, UDP                  | Port number (16-bit)    | Segment / Datagram |
| Network     | IP (IPv4/IPv6), ICMP      | IP address (32/128-bit) | Packet             |
| Link        | Ethernet, Wi-Fi           | MAC address (48-bit)    | Frame              |

**Remark.**

**Encapsulation:** when data travels down the stack, each layer wraps the payload with its own header. On the receiver side, each layer strips its header and passes the payload up. Think of nested envelopes.

# Chapter 12

## Application Layer

### 12.1 HTTP and the Web

#### Definition 12.1.1: HTTP (HyperText Transfer Protocol)

The protocol used by browsers and web servers to exchange web resources. It is a **text-based, stateless, request-response** protocol that runs over TCP (typically port 80 for HTTP, 443 for HTTPS).

#### Request and Response

An HTTP **request** has:

- **Method:** GET (retrieve), POST (create/submit), PUT (update), DELETE (remove), PATCH (partial update).
- **URL:** identifies the resource (e.g., `/api/users/42`).
- **Headers:** metadata (Content-Type, Authorization, Cookie, ...).
- **Body** (optional): data sent with POST/PUT.

An HTTP **response** has:

- **Status code:** 200 OK, 201 Created, 301 Moved Permanently, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Internal Server Error.
- **Headers:** Content-Type, Content-Length, Set-Cookie, ...
- **Body:** HTML, JSON, binary data, ...

| Version  | Key Feature                                                     |
|----------|-----------------------------------------------------------------|
| HTTP/1.0 | One TCP connection per request (non-persistent; high overhead)  |
| HTTP/1.1 | Persistent connections (keep-alive); pipelining                 |
| HTTP/2   | Binary framing; multiplexing multiple streams on one connection |
| HTTP/3   | Runs over QUIC (UDP-based); lower latency, especially on mobile |

#### Remark.

HTTPS = HTTP + **TLS** (Transport Layer Security). TLS encrypts the payload so it cannot be read or tampered with by intermediate nodes. It is triggered by the `https://` scheme; the server presents a **certificate** signed by a trusted Certificate Authority (CA).

## REST APIs

In ML/DS practice, you interact with web services through REST APIs, which are simply HTTP services that follow a set of conventions:

- Resources are identified by URLs (e.g., `/datasets/123`).
- CRUD operations map to HTTP methods: Create → POST, Read → GET, Update → PUT/PATCH, Delete → DELETE.
- Responses are almost always **JSON**.
- Stateless: each request is self-contained (auth token in every request).

## 12.2 DNS

### Definition 12.2.1: DNS (Domain Name System)

A distributed, hierarchical database that translates human-readable **domain names** (e.g., `nyu.edu`) into **IP addresses** (e.g., `128.122.119.209`). Often called the “phone book of the Internet.”

### Resolution Process

1. Your application asks the OS for the IP of `www.example.com`.
2. The OS queries a **recursive resolver** (your ISP’s or a public one like `8.8.8.8`).
3. The resolver queries the **root name servers** (→ finds TLD server for `.com`).
4. The TLD server points to `example.com`’s **authoritative name server**, which returns the IP.
5. The resolver caches the answer for a TTL (time-to-live) period and returns the IP to your OS.

#### Remark.

DNS queries use **UDP** on port 53 (fast; queries are small). DNS **caching** at every level (resolver, OS, browser) reduces latency; TTL values control how long entries stay cached.



# Chapter 13

## Transport Layer

### 13.1 TCP vs. UDP

#### Definition 13.1.1: Multiplexing via Ports

A **port** is a 16-bit number (0–65535) identifying a specific process on a host. Ports allow multiple applications to share one IP address simultaneously. Well-known ports ( $< 1024$ ) are reserved: HTTP=80, HTTPS=443, DNS=53, SSH=22, FTP=21. Client processes use ephemeral ports ( $\geq 1024$ ).

Both TCP and UDP provide multiplexing and a basic checksum for error detection. Beyond that, they differ fundamentally:

|                    | UDP                                         | TCP                                               |
|--------------------|---------------------------------------------|---------------------------------------------------|
| Full name          | User Datagram Protocol                      | Transmission Control Protocol                     |
| Connection         | Connectionless                              | Connection-oriented (3-way handshake)             |
| Delivery           | Best-effort; packets may be lost, reordered | Reliable, in-order byte stream                    |
| Error recovery     | None (application must handle)              | Retransmission, sequence numbers                  |
| Flow control       | None                                        | Yes (prevents fast sender flooding slow receiver) |
| Congestion control | None                                        | Yes (backs off when network is congested)         |
| Overhead           | 8-byte header                               | 20-byte header (minimum)                          |
| Use cases          | DNS, video streaming, gaming, VoIP          | HTTP(S), email, SSH, file transfer                |

#### TCP Reliability Mechanisms

| Mechanism                     | Purpose                                                              |
|-------------------------------|----------------------------------------------------------------------|
| <b>3-way handshake</b>        | Establish connection: SYN → SYN-ACK → ACK                            |
| <b>Sequence numbers</b>       | Number every byte; detect gaps and reorder out-of-order segments     |
| <b>Acknowledgements (ACK)</b> | Receiver confirms bytes received; sender retransmits on timeout      |
| <b>Checksum</b>               | Detect bit errors in the header and payload                          |
| <b>Flow control (window)</b>  | Receiver advertises how much buffer space it has; sender respects it |
| <b>Congestion control</b>     | Sender reduces rate when it detects network congestion               |
| <b>4-way teardown</b>         | FIN → ACK → FIN → ACK                                                |

**Remark.**

The choice between TCP and UDP comes down to one question: does your application tolerate loss? “*Better never than late*” describes real-time video: a delayed video frame is useless, so dropping it (UDP) beats retransmitting it (TCP). “*Never lose a byte*” describes file downloads: TCP is correct.

## 13.2 Ports and Sockets

**Definition 13.2.1: Socket**

A **socket** is one endpoint of a bidirectional communication link. It is identified by the pair (IP address, port number). A TCP connection is identified by *four values*: (source IP, source port, destination IP, destination port).

**Example 13.2.1**

When your browser loads `https://nyu.edu`: source = (your IP, 54321), destination = (128.122.119.209, 443). The server can handle thousands of simultaneous connections because each has a unique source port.

In Python, the `socket` module provides low-level access; `requests` and `httpx` wrap it for HTTP; `urllib` is in the standard library.

# Chapter 14

## Network Layer

### 14.1 IP Addressing and Subnets

#### Definition 14.1.1: IPv4 Address

A **32-bit** number written in **dotted-quad** notation: four decimal octets separated by dots (e.g., 192.168.1.42).  $2^{32} \approx 4.3$  billion unique addresses.

An IPv4 address consists of two parts:

- **Network prefix (netid)**: identifies the network.
- **Host identifier (hostid)**: identifies the specific host within that network.

#### Definition 14.1.2: CIDR (Classless Inter-Domain Routing)

The prefix length is specified with **slash notation**: **a.b.c.d/n** means the first  $n$  bits are the network prefix. Example: 192.168.1.0/24 is a network whose first 24 bits are fixed; the last 8 bits identify hosts. That network has  $2^8 = 256$  addresses (254 usable: all-zeros = network address, all-ones = broadcast address).

| Prefix | Mask            | Addresses                     | Usable hosts        |
|--------|-----------------|-------------------------------|---------------------|
| /8     | 255.0.0.0       | $2^{24} \approx 16.8\text{M}$ | large ISP / Class A |
| /16    | 255.255.0.0     | $2^{16} = 65536$              | university campus   |
| /24    | 255.255.255.0   | $2^8 = 256$                   | small office LAN    |
| /30    | 255.255.255.252 | 4                             | point-to-point link |

#### Special Addresses

| Range           | Purpose                                           |
|-----------------|---------------------------------------------------|
| 127.0.0.0/8     | Loopback (127.0.0.1 = “localhost”)                |
| 0.0.0.0         | Unspecified (used before DHCP assigns an address) |
| 255.255.255.255 | Limited broadcast                                 |
| 10.0.0.0/8      | Private (RFC 1918); not routed on public Internet |
| 172.16.0.0/12   | Private (RFC 1918)                                |
| 192.168.0.0/16  | Private (RFC 1918) — home routers use this        |
| 169.254.0.0/16  | Link-local (auto-assigned when DHCP fails)        |

**Remark.**

**NAT (Network Address Translation):** your home router assigns private addresses (`192.168.x.x`) to your devices and translates them to a single public IP when traffic leaves. This is why you can have many devices sharing one ISP-assigned IP address.

## 14.2 Routing Overview

### Definition 14.2.1: Forwarding

Each router maintains a **forwarding table** that maps destination IP prefixes to outgoing interfaces. On receiving a packet, the router performs **longest-prefix match**: it selects the most specific matching entry.

#### Example 14.2.1

Forwarding table entry `12.34.158.0/21 → interface 3` means any packet whose destination IP starts with the 21-bit prefix `12.34.158` (roughly) exits on interface 3.

Routing protocols compute these tables:

| Protocol | Scope        | Algorithm                                            |
|----------|--------------|------------------------------------------------------|
| RIP      | Intra-domain | Distance-vector (Bellman-Ford)                       |
| OSPF     | Intra-domain | Link-state (Dijkstra)                                |
| BGP      | Inter-domain | Path-vector; the “glue” of the Internet between ISPs |

**Remark.**

You do not need to implement routing protocols as an MLE/DS, but understanding prefixes and longest-prefix match explains why `traceroute` shows the path your packets take, and why cloud regions have different latencies.

## 14.3 IPv6 Basics

IPv4 addresses ( $2^{32} \approx 4.3 \times 10^9$ ) were officially exhausted by IANA in 2011. **IPv6** solves this:

|                 | IPv4                                 | IPv6                                     |
|-----------------|--------------------------------------|------------------------------------------|
| Address size    | 32 bits                              | 128 bits                                 |
| Total addresses | $\approx 4.3 \times 10^9$            | $\approx 3.4 \times 10^{38}$             |
| Notation        | Dotted-quad ( <code>1.2.3.4</code> ) | Hex groups ( <code>2001:0db8::1</code> ) |
| Header size     | 20 bytes (variable)                  | 40 bytes (fixed)                         |
| NAT needed?     | Yes (address exhaustion)             | No                                       |

# Chapter 15

## Link Layer (Brief)

### 15.1 MAC Addresses and Ethernet

#### Definition 15.1.1: MAC Address

A **48-bit** hardware address burned into a network interface card (NIC), written in hex (e.g., `00:1A:2B:3C:4D:5E`). Unlike IP addresses (logical, reassignable), MAC addresses identify a specific physical interface.

The **link layer** is responsible for delivering **frames** between two nodes on the *same* local network segment — it does not cross routers.

**Ethernet** is the dominant wired LAN technology:

- Frames carry up to **1500 bytes** of payload (the Maximum Transmission Unit, MTU).
- Each frame has a source MAC, destination MAC, EtherType, payload, and CRC.
- Higher speeds: 100 Mbps (Fast Ethernet) → 1 Gbps → 10 Gbps → 100 Gbps (data centres).

**Remark.**

Wi-Fi (IEEE 802.11) operates at the link layer too — frames use MAC addresses, but the medium is radio rather than copper or fibre. Switches forward frames based on MAC addresses within a LAN; *routers* forward packets based on IP addresses across LANs.

### 15.2 ARP and DHCP

#### Definition 15.2.1: ARP (Address Resolution Protocol)

Maps an **IP address** → **MAC address** within a LAN. When host A wants to send to IP `192.168.1.5` but does not know its MAC, A broadcasts: “Who has `192.168.1.5`? Tell `AA:BB:...`” The owner replies with its MAC, and A caches the mapping.

**Definition 15.2.2: DHCP (Dynamic Host Configuration Protocol)**

Automatically configures a new host with an **IP address**, subnet mask, default gateway, and DNS server. Uses UDP (server port 67, client port 68). The four-step process:

1. **DHCP Discover** — client broadcasts “I need an IP address.”
2. **DHCP Offer** — server offers an IP and lease duration.
3. **DHCP Request** — client accepts the offer.
4. **DHCP ACK** — server confirms; client configures itself.

**Remark.**

Every time you connect your laptop to a new Wi-Fi network, DHCP runs silently in the background: it sends 0.0.0.0 as source (no IP yet) and broadcasts to 255.255.255.255. Within milliseconds you have an IP, gateway, and DNS resolver — ready to browse.